QuestDB

QuestDB: Architecture and Core Concepts

Technical Deep Dive

Contents

Introduction . . . 2
Architecture overview . . . 2
- Three-tier storage model . . . 2
- Column-oriented storage . . . 3
- Partitions . . . 3
- Data lifecycle . . . 4
- Query engine . . . 4

Data model . . . 4
- Data types . . . 4
- N-dimensional arrays . . . 5
- The SYMBOL type . . . 5
- Designated timestamp . . . 5
- Deduplication . . . 6

SQL extensions . . . 6
- SAMPLE BY . . . 7
- LATEST ON . . . 7
- Time-series joins . . . 8
- TICK interval syntax . . . 9
- Views . . . 9
- Materialized views . . . 9

AI and agent integration . . . 10
Enterprise capabilities . . . 11
- Replication . . . 11
- Security . . . 11
- Deployment options . . . 11

Further reading . . . 11

Introduction

QuestDB is an open source time-series database built from scratch for low-latency ingestion and analytics. Its core is written in zero-GC Java with focused C++ and Rust components, producing a compact codebase optimized for cache locality and predictable tail latency. The storage engine uses column-oriented, time-partitioned files with memory-mapped I/O and vectorized (SIMD) execution to support high-throughput writes and sub-millisecond analytical queries over billions of rows.

SQL is the primary interface, extended with time-series operators such as SAMPLE BY, LATEST ON, ASOF JOIN, and HORIZON JOIN. Data is stored in QuestDB's native binary format for recent data and in Parquet for older partitions, queryable through a single SQL engine. This three-tier storage model combines write performance with open-format interoperability, eliminating vendor lock-in while keeping dataframe libraries and AI frameworks connected natively.

QuestDB's standard interfaces (SQL, PGWire, REST, Parquet, Arrow) make it a natural foundation for AI agents and LLM-powered applications that need fast, auditable access to time-series data.

This document covers QuestDB's internal architecture, data model, SQL extensions, AI integration, and enterprise capabilities.

Architecture overview

Three-tier storage model

QuestDB stores data in three tiers that balance write throughput, query latency, and long-term interoperability.

Tier one: Parallel Write-Ahead Log. All changes are recorded in WAL files before being written to database files, enabling crash recovery by replaying log entries. WAL files are separated per table and per active connection, allowing concurrent data ingestion, modifications, and schema changes without locking. A component called the Sequencer allocates unique transaction numbers chronologically and ensures data consistency across all readers during ongoing write operations.

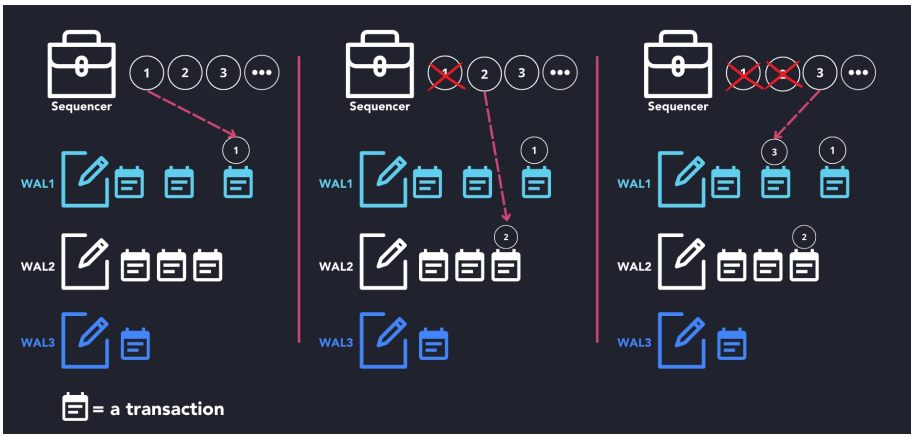

Figure 1: The sequencer allocates unique transaction numbers and orders concurrent writes

Tier two: QuestDB binary table storage. The TableWriter merges WAL data into colum-

nar binary format, handling out-of-order data resolution and deduplication. Column files use an append model. The active (most recent) partition for each table is always stored in this tier for minimum query latency.

Tier three: Parquet. Partitions can be converted to Parquet for both interoperability and compression. This applies to any partition, not only older ones. Partitions in Parquet format remain fully available for queries through the same SQL interface, and users do not need to know whether a partition is in binary or Parquet format. In QuestDB Enterprise, Parquet partitions can reside on object storage (S3, Azure Blob, GCS) or NFS while remaining queryable by the SQL engine. Tables can be configured to convert to Parquet automatically using storage policies.

Column-oriented storage

The system stores each table as separate files per column. Fixed-size data types use one file per column, while variable-size types (such as VARCHAR) use two files. Columnar storage improves CPU utilization during vectorized operations, speeding up aggregations and computations.

Columns with fixed-size data types are read by translating the record number into a file offset by a simple bit shift. The offset is then translated into an offset in a lazily mapped memory page, where the required value is read from. This makes random access to any row in any column a constant-time operation.

By default, QuestDB relies on OS-level durability, letting the operating system write dirty pages to disk. For stronger guarantees, sync commit mode invokes fsync() on each commit, ensuring data survives OS crashes or power loss at the cost of reduced write throughput.

Partitions

QuestDB partitions tables by time intervals, storing each interval's data in a separate directory on disk. Available intervals are HOUR, DAY, WEEK, MONTH, and YEAR.

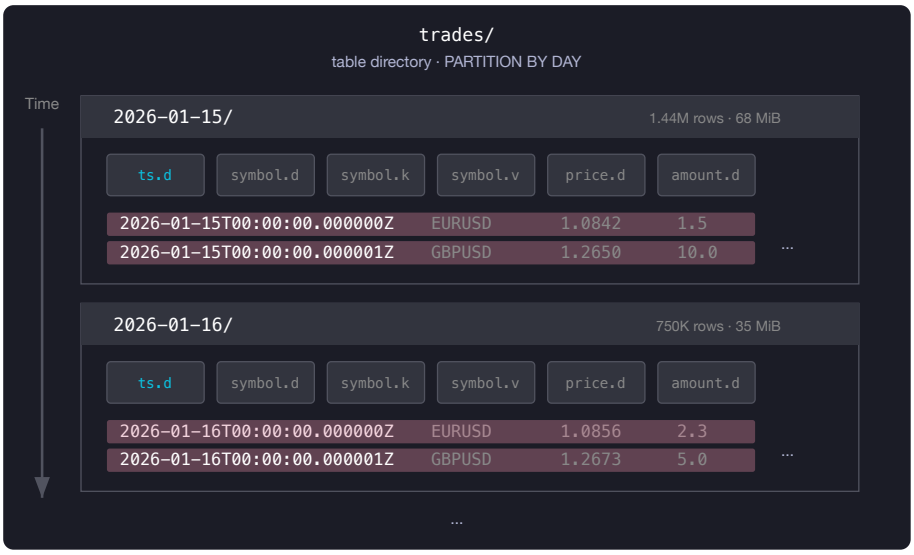

Figure 2: How table data is organized into time-based partition directories, each containing column files

The SQL optimizer skips partitions outside a query's time range, so a query for "last hour" on a table with years of data reads only one partition. Old data can be dropped instantly with DROP PARTITION rather than expensive DELETE operations. Out-of-order data only rewrites affected partitions, not the entire table. Different partitions can be written and read simultaneously without contention.

Each partition directory contains one data file per column (`.d` extension), plus index files for SYMBOL columns. The general guideline is to target 30-80 million rows per partition. When out-of-order data arrives for an existing partition, QuestDB may split it into sub-partitions to avoid rewriting all existing data. Splits are automatically squashed in the background.

Data lifecycle

QuestDB provides two mechanisms for automatic data lifecycle management.

TTL (Time To Live), available in QuestDB Open Source, automatically drops old partitions based on data age. A partition is dropped when its entire time range falls outside the TTL window. The reference time is capped at wall-clock time by default to prevent accidental data loss from far-future timestamps.

Storage policies, available in QuestDB Enterprise, replace TTL with graduated lifecycle management. A storage policy defines stages: TO PARQUET (convert from native binary to Parquet), DROP NATIVE (remove binary files, keeping Parquet), and DROP LOCAL (remove all local data). Each stage has its own independent TTL, and the stages must be ordered chronologically. QuestDB checks storage policies periodically and processes eligible partitions automatically.

Query engine

The query engine includes a custom SQL parser, a just-in-time (JIT) compiler, and a vectorized execution engine. SQL queries are converted into an optimized abstract syntax tree through a compilation pipeline that pushes down predicates and rewrites queries to remove unnecessary operations.

Queries with WHERE clauses compile critical parts of the execution plan to native machine code using SIMD AVX-2 instructions at runtime. Vectorized instructions apply the same operation to many data elements simultaneously, maximizing CPU cache utilization. Multi-threaded execution distributes work across cores, with some queries (such as GROUP BY and SAMPLE BY) executing a pipeline with both single-threaded and multi-threaded stages to avoid slowdowns when groups are unbalanced.

The engine uses the OS page cache to keep frequently accessed data in memory, avoiding garbage collection overhead through off-heap memory managed via memory mapping and direct allocation. Query plans are cached for reuse within the same connection. Configurable worker pools handle specialized functions: parsing incoming data, applying WAL changes, handling PGWire protocol, and responding to HTTP connections.

Data model

Data types

QuestDB provides a rich type system designed for time-series and financial workloads:

Numeric types. `BOOLEAN`, `BYTE`, `SHORT`, `INT`, `LONG`, `FLOAT`, `DOUBLE`, and `LONG256` (unsigned 256-bit, suitable for crypto addresses and hash codes). The `DECIMAL` type provides exact decimal arithmetic with user-specified precision and scale, performing only about 2x slower than DOUBLE, with non-allocating computations.

Time types. `TIMESTAMP` stores microsecond-resolution offsets from Unix epoch. `TIMESTAMP_NS` stores nanosecond-resolution offsets for high-frequency trading and scientific data. `DATE` stores millisecond-resolution offsets but is not recommended for new tables. `INTERVAL` represents a pair of timestamps (expression-only, not persisted).

String types. `VARCHAR` is the recommended string type, using UTF-8 encoding with a 128-bit header. Strings shorter than 9 bytes are fully inlined within the header and occupy no additional data space. `STRING` uses UTF-16 and is maintained only for backward compatibility.

Specialized types. `SYMBOL` uses dictionary encoding for repeated string values (covered in detail below). `UUID` stores 128-bit identifiers efficiently. `IPV4` stores IP addresses as 32-bit integers. `GEOHASH` stores geospatial hashes with configurable precision.

N-dimensional arrays

QuestDB supports N-dimensional arrays of DOUBLE values as a native column type. Arrays are stored as a compact header (20 + 4 bytes per dimension) followed by a dense payload of values. This makes them suitable for storing order book data (price and volume at multiple levels), sensor readings across channels, or any multi-dimensional numeric data that belongs with each row.

For example, the `market_data` table stores full order books as 2D arrays: `bids[1]` holds bid prices, `bids[2]` holds bid volumes, and the same structure applies to `asks`. Array elements can be accessed by index, and arrays can be passed to specialized functions like L2PRICE directly in SQL, without unpacking into separate tables or columns.

The SYMBOL type

SYMBOL is a key optimization for time-series workloads. Internally, each unique string is stored once in a lookup table, and rows store integer references.

Filtering on a SYMBOL column performs integer comparison instead of string comparison. Grouping uses integer grouping instead of string hashing. Storage is lower because strings are stored once rather than repeated per row. Symbol columns can also be indexed for even faster lookups in WHERE clauses. Symbol capacity scales automatically as new values are added.

Designated timestamp

Every time-series table should have a designated timestamp column. This column defines the time axis and unlocks QuestDB's core capabilities: partitioning, SAMPLE BY, LATEST ON, ASOF JOIN, TTL, deduplication, and replication.

The designated timestamp fundamentally changes how QuestDB stores and queries data. Rows are stored physically sorted by this column. When a query includes a timestamp predicate, the engine uses partition pruning to skip entire partitions outside the time range, then binary search within relevant partitions to find exact row boundaries, then reads only the matching data frames from other columns. Most queries with timestamp predicates complete in sub-millisecond time regardless of total table size.

The designated timestamp can be backed by a `TIMESTAMP` column (microsecond resolution) or a `TIMESTAMP_NS` column (nanosecond resolution). It cannot be NULL, cannot be changed after table creation, and cannot be modified with UPDATE. Each table can have at most one, though additional timestamp columns can be stored as regular columns. All timestamps are stored in UTC internally.

Deduplication

Deduplication ensures that only one row exists for a given set of key columns. When a new row matches an existing row's keys, the old row is replaced. When the incoming row is identical, QuestDB skips the write entirely, producing no disk I/O. This significantly reduces write amplification when reloading large datasets where only a small portion has changed.

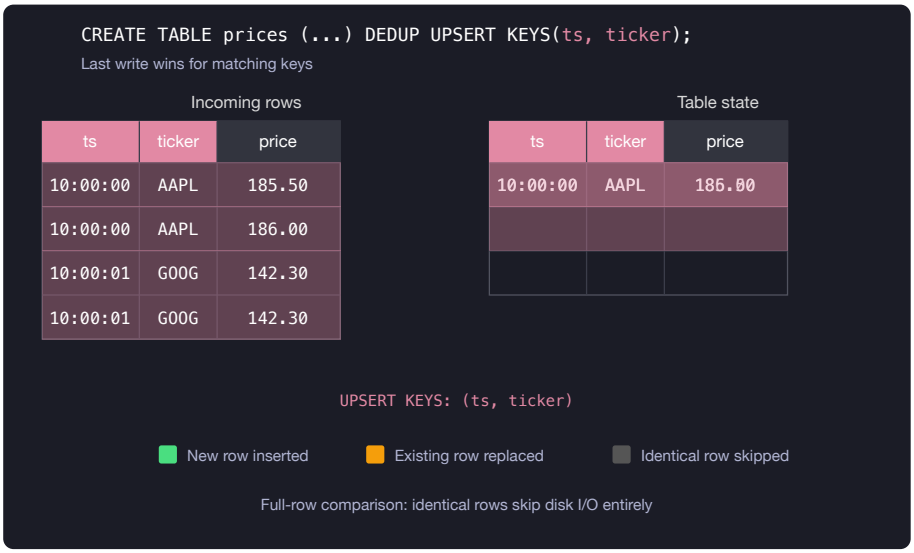

Figure 3: How deduplication handles incoming rows: inserting new keys, replacing duplicates, and skipping identical rows

Deduplication is configured at table creation with DEDUP UPSERT KEYS, which must include the designated timestamp:

```
CREATE TABLE trades (
  timestamp TIMESTAMP,
  symbol SYMBOL,
  price DOUBLE,
  amount DOUBLE
) TIMESTAMP(timestamp)
PARTITION BY DAY
TTL 1 WEEK
DEDUP UPSERT KEYS (timestamp, symbol);
```

SQL extensions

QuestDB uses standard SQL with extensions for time-series workloads. These cover new keywords, join types, aggregate functions, data types, and interval syntax that eliminate the multi-step workarounds general-purpose databases require for common time-series patterns.

SAMPLE BY

SAMPLE BY aggregates data into fixed time buckets. It replaces the GROUP BY + date_trunc pattern used in other databases with a single, optimized clause. The following query generates 1-minute OHLC (Open, High, Low, Close) bars from raw tick data:

```
SELECT
  timestamp,
  symbol,
  first(price) AS open,
  max(price) AS high,
  min(price) AS low,
  last(price) AS close,
  sum(quantity) AS total_volume
FROM fx_trades
WHERE timestamp IN '$today'
SAMPLE BY 1m;
```

SAMPLE BY supports fill strategies for missing intervals (NONE, NULL, PREV, LINEAR, constant value).

LATEST ON

LATEST ON returns the most recent row for each value of a partition key, using the designated timestamp's sorted order. It is the standard pattern for "current state" queries on time-series data:

```
SELECT timestamp, symbol, bids, asks
FROM market_data
WHERE timestamp IN '$today'
LATEST ON timestamp PARTITION BY symbol
```

Combined with QuestDB's native 2D array types for order book data and the L2PRICE function, LATEST ON enables real-time liquidity analysis across instruments:

```
WITH latest_books AS (
  SELECT timestamp, symbol, bids, asks
  FROM market_data
  WHERE timestamp IN '$today'
  LATEST ON timestamp PARTITION BY symbol
)
SELECT
  symbol,
  L2PRICE(100_000, asks[2], asks[1]) AS buy_price,
  L2PRICE(100_000, bids[2], bids[1]) AS sell_price,
  L2PRICE(100_000, asks[2], asks[1])
    - L2PRICE(100_000, bids[2], bids[1])
    AS effective_spread,
  (L2PRICE(100_000, asks[2], asks[1])
    - L2PRICE(100_000, bids[2], bids[1]))
    / ((L2PRICE(100_000, asks[2], asks[1])
      + L2PRICE(100_000, bids[2], bids[1])) / 2)
    * 10_000 AS spread_bps
FROM latest_books
ORDER BY spread_bps;
```

L2PRICE calculates the average execution price when filling against multiple price levels in the order book. The 2D arrays (bids[][]/asks[][]) store the full order book natively, where the first dimension holds prices and the second holds volumes.

Time-series joins

In addition to standard SQL joins (INNER, LEFT, RIGHT, FULL, CROSS, LATERAL), QuestDB provides specialized join types for time-series data:

ASOF JOIN matches each row to the most recent row in the right table by timestamp. It is the standard tool for point-in-time analysis: matching trades to quotes, events to state, or any pattern where you need "what was the value at this moment?" **LT JOIN** and **SPLICE JOIN** offer finer control over how rows are matched by timestamp: LT JOIN matches strictly earlier rows (excluding exact timestamp matches), while SPLICE JOIN interleaves rows from both tables by timestamp.

WINDOW JOIN matches each row to all rows in the right table within a configurable time window, used for aggregation over sliding time ranges.

HORIZON JOIN evaluates the right table at multiple time offsets relative to each left-table row, designed for post-trade analysis. The following query builds a markout curve measuring how the market mid-price moves after each trade, with readings every 5 seconds for 30 seconds:

```
SELECT
    t.symbol,
    t.ecn,
    t.counterparty,
    t.passive,
    h.offset / 1000000000 AS horizon_sec,
    count() AS n,
    avg(
        CASE t.side
            WHEN 'buy'  THEN ((m.best_bid + m.best_ask) / 2 - t.price)
                             / t.price * 10000
            WHEN 'sell' THEN (t.price - (m.best_bid + m.best_ask) / 2)
                             / t.price * 10000
        END
    ) AS avg_markout_bps,
    sum(
        CASE t.side
            WHEN 'buy'  THEN ((m.best_bid + m.best_ask) / 2 - t.price)
                             * t.quantity
            WHEN 'sell' THEN (t.price - (m.best_bid + m.best_ask) / 2)
                             * t.quantity
        END
    ) AS total_pnl
FROM fx_trades t
HORIZON JOIN market_data m ON (symbol)
    RANGE FROM 0s TO 30s STEP 5s AS h
WHERE t.timestamp IN '$now-1h..$now'
GROUP BY t.symbol, t.ecn, t.counterparty, t.passive, horizon_sec
ORDER BY t.symbol, t.ecn, t.counterparty, t.passive, horizon_sec;
```

A positive markout means the trade was profitable in hindsight. A persistently negative markout against specific counterparties signals adverse selection and may warrant flow management.

TICK interval syntax

TICK provides a concise way to define time ranges. It generates UTC intervals at query planning time, enabling binary search on sorted data rather than row-by-row evaluation:

```
-- Last hour of data
WHERE ts IN '$now - 1h..$now'

-- Last 30 minutes
WHERE ts IN '$now - 30m..$now'

-- Today's data (full day)
WHERE ts IN '$today'

-- Last 5 business days
WHERE ts IN '$today - 5bd..$today - 1bd'

-- All workdays in January at 09:00 for 8 hours
WHERE ts IN '[2024-01]T09:00#workday;8h'

-- All of 2024 at 09:30
WHERE ts IN '[2024]T09:30'

-- Multiple times on one day
WHERE ts IN '2024-01-15T[09:00,12:00,18:00];1h'

-- With timezone
WHERE ts IN '2024-01-15T09:30@America/New_York;6h30m'
```

TICK handles DST transitions automatically. QuestDB Enterprise extends TICK with exchange calendars, which replace manual specification of trading hours, day filters, and holidays with real exchange schedules referenced by ISO 10383 MIC code:

```
-- NYSE regular trading hours for January, holidays excluded automatically
SELECT * FROM trades
WHERE ts IN '[2025-01]#XNYS';
```

Views

A view is a virtual table defined by a SQL SELECT statement. Views are fully transparent to the query optimizer: filter push-down, projection push-down, join optimization, and partition pruning all work across view boundaries with no performance penalty.

Views support parameterized queries through the DECLARE statement. Parameters marked as OVERRIDABLE can be changed at query time. In QuestDB Enterprise, views use a definer security model: users querying the view need only SELECT permission on the view, not on the underlying tables.

Materialized views

A materialized view stores pre-computed results of a SAMPLE BY query. When new data arrives in the base table, only the affected time slices are recomputed incrementally. This makes materialized views ideal for powering real-time workflows like live dashboards: many concurrent users can query the view simultaneously, but the aggregation runs only once per new batch of data rather than once per query.

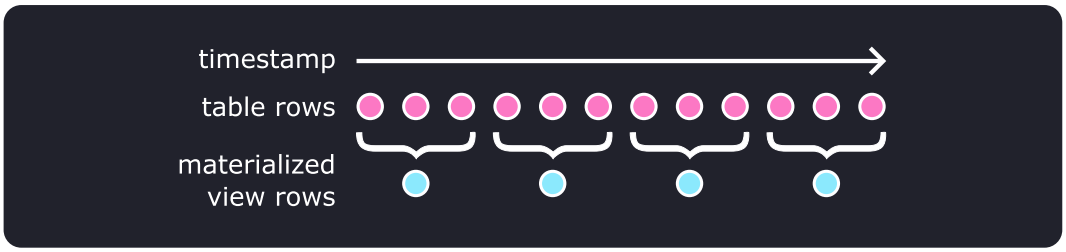

Figure 4: How raw data is sampled into a materialized view

QuestDB supports four refresh strategies: IMMEDIATE (the default, updates after each base table transaction), EVERY (timer-based), PERIOD (fixed intervals aligned to a time zone), and MANUAL (explicit SQL trigger). Materialized views use the same storage engine as regular tables, with independent partitioning and lifecycle management. They can be chained, with the output of one serving as input to another.

In QuestDB Open Source, materialized views support TTL for automatic data retention:

```
CREATE MATERIALIZED VIEW trades_hourly AS (
  SELECT timestamp, symbol, avg(price) AS avg_price FROM trades SAMPLE BY 1h
) PARTITION BY DAY TTL 7 DAYS;
```

In QuestDB Enterprise, storage policies replace TTL with graduated lifecycle management:

```
CREATE MATERIALIZED VIEW trades_hourly AS (
  SELECT timestamp, symbol, avg(price) AS avg_price FROM trades SAMPLE BY 1h
) PARTITION BY DAY
  STORAGE POLICY(TO PARQUET 7d, DROP NATIVE 14d);
```

Querying a materialized view that pre-computes OHLC bars returns in single-digit milliseconds, compared to hundreds of milliseconds for the equivalent aggregation scanning tens of millions of raw rows.

AI and agent integration

QuestDB integrates with AI coding agents through standard protocols. SQL is the query interface, PGWire provides PostgreSQL-compatible connectivity, and the REST API enables HTTP-based access. As described in the storage architecture, Parquet data can be consumed in-place by dataframe libraries (Polars, Pandas, Spark) and ML pipelines, or exported on demand. No proprietary protocols or vendor SDKs are required.

The QuestDB agent skill for Claude Code and OpenAI Codex embeds SQL syntax, ingestion patterns, Grafana templates, and financial indicator recipes directly into the agent's context. Agents discover the table schema via the REST API, write optimized SQL using QuestDB's time-series extensions, and execute queries returning results in milliseconds. Every agent query is standard SQL, producing deterministic, auditable results. No setup is required to get started: the public demo instance at demo.questdb.io is accessible immediately.

QuestDB's timestamp-ordered, lock-free architecture handles concurrent queries from both applications and agents by design.

Enterprise capabilities

QuestDB Enterprise builds on the open source core with capabilities required for production deployments in regulated environments.

Replication

QuestDB Enterprise provides primary-replica replication for high availability and disaster recovery. The primary instance writes data to the WAL and uploads WAL files to an object store (AWS S3, Azure Blob Storage, Google Cloud Storage, or NFS). Replica instances download and apply these files to stay in sync. No direct network connections are required between nodes.

This decoupled architecture means replicas can be added or removed without touching the primary, can be placed in different regions or availability zones, and the object store provides durability and point-in-time recovery. Two availability strategies are supported: hot availability (continuous replicas for instant failover) and cold availability (reconstruct a new primary from snapshots and WAL files when needed).

Security

TLS encryption secures all network interfaces. Role-Based Access Control (RBAC) provides users, groups, and fine-grained permissions down to column level. Single Sign-On via OpenID Connect (OAuth 2.0/OIDC) integrates with identity providers such as Microsoft Entra ID and PingFederate. Audit logs and secure service accounts support inter-machine communication and compliance requirements.

Deployment options

QuestDB Enterprise is a binary-compatible upgrade from Open Source: download the Enterprise binaries, swap them in, and restart. Existing data works immediately.

For managed deployments, QuestDB offers a Bring Your Own Cloud (BYOC) model. QuestDB's team handles operations of all primary and replica instances on the customer's infrastructure using standard cloud provider tools (CloudFormation for AWS, Lighthouse for Azure). All data resides in the customer's account, meeting data localization requirements for GDPR, FINRA, or internal compliance policies. The infrastructure is fully owned and auditable by the customer.

Further reading

Explore the documentation. The full QuestDB documentation at questdb.com/docs covers SQL reference, ingestion guides with client libraries for Python, Go, Java, Rust, C/C++, and .NET, configuration reference, and operational procedures.

Try QuestDB. QuestDB Open Source is available on Linux, macOS, Windows, Docker, and Kubernetes. Download and start a local instance in minutes for hands-on testing with your own data. A live demo instance at demo.questdb.com provides immediate access to over 2 billion rows of sample data with no installation required.

Evaluate Enterprise. Contact the QuestDB team at questdb.com/enterprise/contact for architecture reviews and guided deployment with Enterprise features including high availability, RBAC, TLS, and tiered storage.

Scan to explore the documentation